

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4
дисциплины «Искусственный интеллект и машинное обучение »
Вариант №9

Выполнила:

Ковжого Елизавета Андреевна

2 курс, группа ИВТ-б-о-24-1,

09.03.01 «Информатика и вычислительная техника», направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

(подпись)

Проверил:

Воронкин Р. А., доцент
департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры Series и базовые операции.

Цель: познакомиться с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Порядок выполнения работы:

Адрес репозитория: https://github.com/LissKovzogo/AI_ML_LAB_4.git

1. Создали, настроили и клонировали репозиторий AI_ML_LAB_4.
2. Создали виртуальное окружение в VisualStudio Code и создали в JupyterLab.
3. Проработали все примеры лабораторной работы:
Поработали со структурами Series, с их значениями и индексами

```
[3]: s1 = pd.Series([1, 2, 3, 4, 5])
      print(s1)

0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
[4]: s2 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
      print(s2)

a    1
b    2
c    3
d    4
e    5
dtype: int64
```

```
[5]: ndarr = np.array([1, 2, 3, 4, 5])
      type(ndarr)

numpy.ndarray
```

```
[6]: s3 = pd.Series(ndarr, ['a', 'b', 'c', 'd', 'e'])
      print(s3)

a    1
b    2
c    3
d    4
e    5
dtype: int64
```

```
[7]: d = {'a':1, 'b':2, 'c':3}
      s4 = pd.Series(d)
      print(s4)

a    1
b    2
c    3
dtype: int64
```

```
[8]: a = 7
      s5 = pd.Series(a, ['a', 'b', 'c'])
      print(s5)

a    7
b    7
c    7
dtype: int64
```

Рисунок 1. Работа с Series

```
[12]: s6 = pd.Series([1, 2, 3, 4, 5], ['a', 'b', 'c', 'd', 'e'])
      print(s6['c'])
      print(s6.iloc[2])

      3
      3

[13]: print(s6['d'])

      4

[14]: print(s6[:2])

      a    1
      b    2
      dtype: int64

[20]: s6[s6<=3]

[20]: a    1
      b    2
      c    3
      dtype: int64

[22]: s7 = pd.Series([10, 20, 30, 40, 50], ['a', 'b', 'c', 'd', 'e'])
      print(s6 + s7)
      print(s6*3)

      a    11
      b    22
      c    33
      d    44
      e    55
      dtype: int64
      a     3
      b     6
      c     9
      d    12
      e    15
      dtype: int64

[23]: s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
      print(s.iloc[0])
      print(s.iloc[2])
      print(s.iloc[-1])

      10
      30
      50
```

Рисунок 2. Работа с Series

```
[26]: print(s.iloc[1:3])
      print(s.loc["b":"d"])

      b    20
      c    30
      dtype: int64
      b    20
      c    30
      d    40
      dtype: int64

[29]: s = pd.Series([10,25,8,30,15], index=['a','b','c','d','e'])
      filtered_s=s[s>10]
      print(filtered_s)

      b    25
      d    30
      e    15
      dtype: int64

[30]: print(s>10)

      a    False
      b     True
      c    False
      d     True
      e     True
      dtype: bool

[31]: filtered_s=s[(s>=10)&(s<=30)]
      print(filtered_s)

      a    10
      b    25
      d    30
      e    15
      dtype: int64

[32]: filtered_s = s[s.index.isin(['b','d'])]
      print(filtered_s)

      b    25
      d    30
      dtype: int64

[34]: s_with_nan=pd.Series([10,None,8,30,None],index=['a','b','c','d','e'])
      filtered_s=s_with_nan[s_with_nan.notnull()]
      print(filtered_s)

      a    10.0
      c     8.0
      d    30.0
      dtype: float64
```

Рисунок 3. Работа с Series

```
s=pd.Series([10,20,30,40],index=['a','b','c','d'])
s.loc['b']=25
print(s)
```

```
a    10
b    25
c    30
d    40
dtype: int64
```

```
s=pd.Series([10,20,30,40],index=['a','b','c','d'])
s[s>30]+=10
print(s)
```

```
a    10
b    20
c    30
d    50
dtype: int64
```

```
s.loc[['a','c']]=[100,200]
print(s)
```

```
a    100
b     20
c    200
d     50
dtype: int64
```

```
s = s.apply(lambda x:x*2)
print(s)
```

```
a    200
b     40
c    400
d    100
dtype: int64
```

```
s_with_nan=pd.Series([10,None,8,30,None],index=['a','b','c','d','e'])
s_filled = s_with_nan.fillna(0)
print(s_filled)
```

```
a    10.0
b     0.0
c     8.0
d    30.0
e     0.0
dtype: float64
```

Рисунок 4. Работа с Series

```
[17]: s=pd.Series([10,20,30,40,50,60,70],index=['a','b','c','d','e','f','g'])
      print(s.head(3))
```

```
a    10
b    20
c    30
dtype: int64
```

```
[18]: print(s.head())
```

```
a    10
b    20
c    30
d    40
e    50
dtype: int64
```

```
[19]: print(s.tail(3))
```

```
e    50
f    60
g    70
dtype: int64
```

```
[20]: print(s.tail())
```

```
c    30
d    40
e    50
f    60
g    70
dtype: int64
```

```
[21]: s=pd.Series([10,20,30,40,50],index=['a','b','c','d','e'])
      print(s.index)
```

```
Index(['a', 'b', 'c', 'd', 'e'], dtype='str')
```

```
[22]: print(s.index[0])
      print(s.index[-1])
```

```
a
e
```

```
[24]: print(s.values)
      print(s.values[0])
      print(s.values.mean())
```

```
[10 20 30 40 50]
10
30.0
```

Рисунок 5. Работа с Series

```
[26]: s=pd.Series([10,20,30,40,50])
      print(s.dtype)

      int64

[27]: s1=pd.Series([1.0,2.5,3.7])
      print(s1.dtype)

      float64

      s2=pd.Series(["apple","bananan","cherry"])
      print(s2.dtype)

      str

      s3=pd.Series([True,False,True])
      print(s3.dtype)

      bool

[28]: s1_int=s1.astype(int)
      print(s1_int)
      print(s1_int.dtype)

      0    1
      1    2
      2    3
      dtype: int64
      int64

[29]: s4=pd.Series([10,"apple",3.14,True])
      print(s4.dtype)

      object

[30]: s = pd.Series([10,np.nan,30,None,50])
      print(s.isnull())

      0    False
      1     True
      2    False
      3     True
      4    False
      dtype: bool

[31]: print(s.notnull())

      0     True
      1    False
      2     True
      3    False
      4     True
      dtype: bool
```

Рисунок 6. Работа с Series


```
[32]: filtered_s = s[s.notnull()]
      print(filtered_s)

0    10.0
2    30.0
4    50.0
dtype: float64
```

```
[33]: mis_s = s[s.isnull()]
      print(mis_s)

1    NaN
3    NaN
dtype: float64
```

```
[37]: s = pd.Series([10, np.nan, 30, None, 50])
      s_filled = s.fillna(0)
      print(s_filled)

0    10.0
1     0.0
2    30.0
3     0.0
4    50.0
dtype: float64
```

```
[39]: s = pd.Series([10, np.nan, 30, None, 50])
      s_filled = s.fillna(s.mean())
      print(s_filled)

0    10.0
1    30.0
2    30.0
3    30.0
4    50.0
dtype: float64
```

```
[41]: s_clean = s.dropna()
      print(s_clean)

0    10.0
2    30.0
4    50.0
dtype: float64
```

```
[42]: s = pd.Series([10, 20, 30, 40, 50])
      s_mult = s*2
      print(s_mult)

0     20
1     40
2     60
3     80
4    100
dtype: int64
```

Рисунок 7. Работа с Series

```
[44]: s1 = pd.Series([1, 2, 3, 4, 5], index = ['a','b','c','d','e'])
      s2 = pd.Series([10, 20, 30, 40, 50], index = ['a','b','c','d','e'])
      s_sum = s1+s2
      print(s_sum)

a    11
b    22
c    33
d    44
e    55
dtype: int64
```

```
[45]: s3 = pd.Series([100,200,300], index = ['a','b','f'])
      s_res = s1+s3
      print(s_res)

a    101.0
b    202.0
c      NaN
d      NaN
e      NaN
f      NaN
dtype: float64
```

```
[46]: s_res = s1.add(s3, fill_value = 0)
      print(s_res)

a    101.0
b    202.0
c      3.0
d      4.0
e      5.0
f    300.0
dtype: float64
```

```
[47]: s = pd.Series([1,2,3,4,5])
      s_squar = s.apply(lambda x:x**2)
      print(s_squar)

0     1
1     4
2     9
3    16
4    25
dtype: int64
```

Рисунок 8. Работа с Series

```
[48]: def cust_func(x):
      return f"Value:{x}"
      s_trans = s.apply(cust_func)
      print(s_trans)

      0    Value:1
      1    Value:2
      2    Value:3
      3    Value:4
      4    Value:5
      dtype: str
```

```
[49]: s_log=s.apply(np.log)
      print(s_log)

      0    0.000000
      1    0.693147
      2    1.098612
      3    1.386294
      4    1.609438
      dtype: float64
```

```
[50]: s_text = pd.Series(["apple", "bananan", "cherry"])
      s_upp = s_text.apply(str.upper)
      print(s_upp)

      0    APPLE
      1  BANANAN
      2   CHERRY
      dtype: str
```

```
[56]: s=pd.Series([10,20,30,40,50])
      print(s.sum())
      print(s.mean())
      print(s.max())
      print(s.min())

      150
      30.0
      50
      10
```

```
[55]: s_with_nan = pd.Series([10,20,None,40,50])
      print(s_with_nan.sum())
      print(s_with_nan.mean())

      120.0
      30.0
```

Рисунок 9. Работа с Series

```
[57]: print(s.describe())

count      5.000000
mean       30.000000
std        15.811388
min        10.000000
25%        20.000000
50%        30.000000
75%        40.000000
max        50.000000
dtype: float64
```

```
[60]: s_text = pd.Series(["apple", "banana", "apple", "cherry", "banana"])
print(s_text.describe())

count      5
unique     3
top        apple
freq       2
dtype: object
```

```
[63]: s = pd.Series([1,2,3,4,5])
s_log = np.log(s)
print(s_log)
print("////////")
s_exp = np.exp(s)
print(s_exp)
print("////////")
s_sqrt = np.sqrt(s)
print(s_sqrt)

0    0.000000
1    0.693147
2    1.098612
3    1.386294
4    1.609438
dtype: float64
////////
0     2.718282
1     7.389056
2    20.085537
3    54.598150
4   148.413159
dtype: float64
////////
0    1.000000
1    1.414214
2    1.732051
3    2.000000
4    2.236068
dtype: float64
```

Рисунок 10. Работа с Series

```
[64]: print(np.sin(s))
      print(np.cos(s))
      print(np.abs(s))

0    0.841471
1    0.909297
2    0.141120
3   -0.756802
4   -0.958924
dtype: float64
0    0.540302
1   -0.416147
2   -0.989992
3   -0.653644
4    0.283662
dtype: float64
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
[65]: s_with_nan= pd.Series([1,np.nan,4,9])
      print(np.sqrt(s_with_nan))

0    1.0
1    NaN
2    2.0
3    3.0
dtype: float64
```

```
[66]: s = pd.Series([1, 2, 3, 4], index = ['a','b','c','d'])
      s.index = ['x','y','z','w']
      print(s)

x    1
y    2
z    3
w    4
dtype: int64
```

```
[67]: s_reset=s.reset_index()
      print(s_reset)

   index  0
0      x  1
1      y  2
2      z  3
3      w  4
```

Рисунок 11. Работа с Series

```
[68]: s_uniq = pd.Series([10,20,30], index=['a','b','c'])
      print(s_uniq.index.is_unique)

      True

[69]: s_dupl=pd.Series([10,20,30], index=['a','b','a'])
      print(s_dupl.index.is_unique)

      False

[70]: print(s_dupl.loc['a'])

      a    10
      a    30
      dtype: int64

[71]: s_reset = s_dupl.reset_index(drop=True)
      print(s_reset)

      0    10
      1    20
      2    30
      dtype: int64

[72]: s_dupl.index = [f"{i}_{n}" for n, i in enumerate(s_dupl.index)]
      print(s_dupl)

      a_0    10
      b_1    20
      a_2    30
      dtype: int64

[74]: s = pd.Series([10,20,30,40], index=['d','b','a','c'])
      s_sorted = s.sort_index()
      print(s_sorted)

      a    30
      b    20
      c    40
      d    10
      dtype: int64

[75]: s = pd.Series([10,20,30,40], index=['d','b','a','c'])
      s_sorted_desc = s.sort_index(ascending=False)
      print(s_sorted_desc)

      d    10
      c    40
      b    20
      a    30
      dtype: int64
```

Рисунок 12. Работа с Series

```
[76]: s_sort_values = s.sort_values()
print(s_sort_values)

d    10
b    20
a    30
c    40
dtype: int64

[77]: s_sort_values_des = s.sort_values(ascending=False)
print(s_sort_values_des)

c    40
a    30
b    20
d    10
dtype: int64

[78]: s_nan = pd.Series([10, None, 30, None, 20], index=['a', 'b', 'c', 'd', 'e'])
print(s_nan.sort_values())

a    10.0
e    20.0
c    30.0
b     NaN
d     NaN
dtype: float64

[79]: print(s_nan.sort_values(na_position='first'))

b     NaN
d     NaN
a    10.0
e    20.0
c    30.0
dtype: float64

[81]: dates=pd.date_range(start='2024-03-01',periods=5,freq='D')
s=pd.Series([100,105,102,98,110], index=dates)
print(s)

2024-03-01    100
2024-03-02    105
2024-03-03    102
2024-03-04     98
2024-03-05    110
Freq: D, dtype: int64
```

Рисунок 13. Работа с Series

```
[82]: print(s['2024-03-03'])
      print(s['2024-03-02':'2024-03-04'])

      102
      2024-03-02    105
      2024-03-03    102
      2024-03-04     98
      Freq: D, dtype: int64
```

```
[86]: dates = pd.date_range(start='2024-03-01', periods=5, freq='h')
      s = pd.Series([10, 15, 12, 8, 20], index=dates)
      print(s)

      2024-03-01 00:00:00    10
      2024-03-01 01:00:00    15
      2024-03-01 02:00:00    12
      2024-03-01 03:00:00     8
      2024-03-01 04:00:00    20
      Freq: h, dtype: int64
```

```
[87]: s = pd.Series([10, 15, 12, ], index=['2024-03-01', '2024-03-02', '2024-03-03'])
      s.index = pd.to_datetime(s.index)
      print(s)
```



```

      2024-03-01    10
      2024-03-02    15
      2024-03-03    12
      dtype: int64
```

```
[88]: print(s.resample('D').mean())

      2024-03-01    10.0
      2024-03-02    15.0
      2024-03-03    12.0
      Freq: D, dtype: float64
```

```
[90]: print(s[s.index.year==2024])

      2024-03-01    10
      2024-03-02    15
      2024-03-03    12
      dtype: int64
```

```
[91]: print(s.shift(1))

      2024-03-01     NaN
      2024-03-02    10.0
      2024-03-03    15.0
      dtype: float64
```

Рисунок 14. Работа с Series


```
[3]: s = pd.Series([10,20,30,40,50,60,70], index=pd.date_range("2024-03-01",
periods=7, freq='D'))
s_ma = s.rolling(window=3).mean()
print(s_ma)
```

2024-03-01	NaN
2024-03-02	NaN
2024-03-03	20.0
2024-03-04	30.0
2024-03-05	40.0
2024-03-06	50.0
2024-03-07	60.0

Freq: D, dtype: float64

```
[94]: s_ma5 = s.rolling(window=5).mean()
print(s_ma5)
```

2024-03-01	NaN
2024-03-02	NaN
2024-03-03	NaN
2024-03-04	NaN
2024-03-05	30.0
2024-03-06	40.0
2024-03-07	50.0

Freq: D, dtype: float64

```
[95]: s_ma2 = s.rolling(window=3, min_periods=1).mean()
print(s_ma2)
```

2024-03-01	10.0
2024-03-02	15.0
2024-03-03	20.0
2024-03-04	30.0
2024-03-05	40.0
2024-03-06	50.0
2024-03-07	60.0

Freq: D, dtype: float64

Рисунок 15. Работа с Series

```
[4]: import matplotlib.pyplot as plt
plt.figure(figsize=(8,4))
plt.plot(s,label=r'Исходные данные', marker="o")
plt.plot(s.rolling(window=3).mean(),label=r'Скользящее среднее (3)', 1
plt.xlabel(r"Дата")
plt.ylabel(r"Значение")
plt.title(r"Скользящее среднее в Series")
plt.legend()
plt.grid()
plt.show()
```



```
[7]: s=pd.Series([100,110,120,90,150])
print(s.pct_change())
```

```
0      NaN
1    0.100000
2    0.090909
3   -0.250000
4    0.666667
dtype: float64
```

```
[10]: print(s.pct_change(periods=2))
```

```
0      NaN
1      NaN
2    0.200000
3   -0.181818
4    0.250000
dtype: float64
```

Рисунок 16. Работа с Series

```
[6]: dates=pd.date_range(start="2024-01-01",periods=30, freq="D")
prices = pd.Series(np.random.randint(90,110, size=30), index=dates)
returns =prices.pct_change()*100
plt.figure(figsize=(10,5))
plt.plot(prices,label=r"Цена акции", marker="o")
plt.ylabel(r"Цена")
plt.xlabel(r"Дата")
plt.title(r"График цены акции")
plt.figure(figsize=(10,5))
plt.bar(returns.index,returns, color="red", alpha=0.7, label=r"")
plt.axhline(0,color='black', linestyle='--')
plt.ylabel(r"Изменение (%)")
plt.xlabel(r"Дата")
plt.title(r"Процентное изменение цены акции")
plt.show()
```



Рисунок 17. Работа с Series

```
[19]: data = {
        'Цена': [100, 250, 180, 320, 450],
        'Товар': ['Яблоки', 'Бананы', 'Апельсины', 'Груши', 'Виноград']
    }
    df = pd.DataFrame(data)
    df.to_csv('data.csv', index=False, encoding='utf-8')

    df = pd.read_csv("data.csv")

    s = df["Цена"]
    print(s)

0    100
1    250
2    180
3    320
4    450
Name: Цена, dtype: int64
```

Рисунок 18. Работа с csv-файлом

2.Выполнили представленные задания:

Задание 1: Создайте Series из списка чисел [5, 15, 25, 35, 45] с индексами ['a', 'b', 'c', 'd', 'e']. Выведите его на экран и определите его тип данных.

Задание 2: Дан Series с индексами ['A', 'B', 'C', 'D', 'E'] и значениями [12, 24, 36, 48, 60]. Используйте .loc[] для получения элемента с индексом 'C' и .iloc[] для получения третьего элемента.

Задание 3: Создайте Series из массива NumPy np.array([4, 9, 16, 25, 36, 49, 64]). Выберите только те элементы, которые больше 20, и выведите результат.

Задание 4: Создайте Series, содержащий 50 случайных целых чисел от 1 до 100 (используйте np.random.randint). Выведите первые 7 и последние 5 элементов с помощью .head() и .tail().

Задание 5: Создайте Series из списка ['cat', 'dog', 'rabbit', 'parrot', 'fish']. Определите тип данных с помощью `.dtype`, затем преобразуйте его в `category` с помощью `.astype()`.

Задание 6: Создайте Series с данными [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]. Напишите код, который проверяет, есть ли в Series пропущенные значения (NaN), и выведите индексы таких элементов.

Задание 7: Используйте Series из предыдущего задания и замените все NaN на среднее значение всех непустых элементов. Выведите результат.

Задание 8: Создайте два Series:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

```
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])
```

Выполните сложение `s1 + s2`. Объясните, почему в результате появляются NaN, и замените их на 0.

Задание 9: Создайте Series из чисел [2, 4, 6, 8, 10]. Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью `.apply(np.sqrt)`.

Задание 10: Создайте Series из 20 случайных чисел от 50 до 150 (используйте `np.random.randint`). Найдите сумму, среднее, минимальное и максимальное значение. Выведите также стандартное отклонение.

Задание 11: Создайте Series, где индексами будут даты с 1 по 10 марта 2024 года (`pd.date_range(start='2024-03-01', periods=10, freq='D')`), а значениями — случайные числа от 10 до 100. Выберите данные за 5–8 марта.

Задание 12: Создайте Series с индексами ['A', 'B', 'A', 'C', 'D', 'B'] и значениями [10, 20, 30, 40, 50, 60]. Проверьте, являются ли индексы

уникальными. Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

Задание 13: Создайте Series, где индексами будут строки ['2024-03-10', '2024-03-11', '2024-03-12'], а значениями [100, 200, 300]. Преобразуйте индексы в DatetimeIndex и выведите тип данных индекса.

Задание 14: Создайте CSV-файл data.csv. Прочитайте файл и создайте Series, используя "Дата" в качестве индекса.

Задание 15: Создайте Series, где индексами будут даты с 1 по 30 марта 2024 года, а значениями — случайные числа от 50 до 150. Постройте график значений с помощью matplotlib. Добавьте заголовок, подписи осей и сетку.

```
import pandas as pd
s = pd.Series([5, 15, 25, 35, 45], ['a', 'b', 'c', 'd', 'e'])
print(s)
print("\nТип данных Series:", s.dtype)
```

```
a      5
b     15
c     25
d     35
e     45
dtype: int64
```

Тип данных Series: int64

Рисунок 2. Задание №1

```
|: s2 = pd.Series([12, 24, 36, 48, 60], ['A', 'B', 'C', 'D', 'E'])
print("Элемент с индексом 'C' (loc):", s2.loc['C'])
print("Третий элемент (iloc):", s2.iloc[2])
```

```
Элемент с индексом 'C' (loc): 36
Третий элемент (iloc): 36
```

Рисунок 20. Задание №2

```

: s3 = pd.Series(np.array([4, 9, 16, 25, 36, 49, 64]))
  result = s3[s3 > 20]
  print("Элементы больше 20:")
  print(result)

```

Элементы больше 20:

3 25

4 36

5 49

6 64

dtype: int64

Рисунок 21. Задание №3

```

: s4 = pd.Series(np.random.randint(1, 101, 50))
  print("Первые 7 элементов:")
  print(s4.head(7))
  print("\nПоследние 5 элементов:")
  print(s4.tail(5))

```

Первые 7 элементов:

0 58

1 35

2 67

3 4

4 24

5 14

6 46

dtype: int32

Последние 5 элементов:

45 35

46 52

47 10

48 93

49 66

dtype: int32

Рисунок 22. Задание №4

```
]: s5 = pd.Series(['cat', 'dog', 'rabbit', 'parrot', 'fish'])
print("Исходный тип:", s5.dtype)
s5_category = s5.astype('category')
print("После преобразования:", s5_category.dtype)
```

Исходный тип: str

После преобразования: category

Рисунок 23. Задание №5

```
: s6 = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])
print("Есть ли пропуски?", s6.isna().any())
print("Индексы с пропущенными значениями:")
print(s6[s6.isna()].index)
```

Есть ли пропуски? True

Индексы с пропущенными значениями:

RangeIndex(start=1, stop=5, step=2)

Рисунок 24. Задание №6

```
s7 = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])
mean_value = s7.mean()
print("Среднее значение:", mean_value)
s7_filled = s7.fillna(mean_value)
print("Результат после замены NaN:")
print(s7_filled)
```

Среднее значение: 4.25

Результат после замены NaN:

0 1.20

1 4.25

2 3.40

3 4.25

4 5.60

5 6.80

dtype: float64

Рисунок 25. Задание №7


```

s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])
result = s1 + s2
print("Результат сложения:")
print(result)
print("\nЗамена NaN на 0:")
result_filled = result.fillna(0)
print(result_filled)

```

Результат сложения:

```

a      NaN
b    25.0
c    45.0
d    65.0
e      NaN
dtype: float64

```

Замена NaN на 0:

```

a      0.0
b    25.0
c    45.0
d    65.0
e      0.0
dtype: float64

```

Рисунок 26. Задание №8

```

s9 = pd.Series([2, 4, 6, 8, 10])
result9 = s9.apply(np.sqrt)
print("Квадратные корни:")
print(result9)

```

Квадратные корни:

```

0      1.414214
1      2.000000
2      2.449490
3      2.828427
4      3.162278
dtype: float64

```

Рисунок 27. Задание №9

```

s10 = pd.Series(np.random.randint(50, 151, 20))
print("Series:", s10.tolist())
print("Сумма:", s10.sum())
print("Среднее:", s10.mean())
print("Минимум:", s10.min())
print("Максимум:", s10.max())
print("Стандартное отклонение:", s10.std())

```

Рисунок 28. Задание №10

```

dates = pd.date_range(start='2024-03-01', periods=10, freq='D')
s11 = pd.Series(np.random.randint(10, 101, 10), index=dates)
print("Весь Series:")
print(s11)
print("\nДанные за 5-8 марта 2024:")
print(s11['2024-03-05':'2024-03-08'])

```

Весь Series:

2024-03-01	84
2024-03-02	39
2024-03-03	71
2024-03-04	75
2024-03-05	55
2024-03-06	73
2024-03-07	48
2024-03-08	10
2024-03-09	40
2024-03-10	93

Freq: D, dtype: int32

Данные за 5-8 марта 2024:

2024-03-05	55
2024-03-06	73
2024-03-07	48
2024-03-08	10

Freq: D, dtype: int32

Рисунок 29. Задание №11

```

: s12 = pd.Series([10, 20, 30, 40, 50, 60], index=['A', 'B', 'A', 'C',
print("Проверка уникальности индексов:", s12.index.is_unique)
print("\nИсходный Series:")
print(s12)
print("\nГруппировка с суммированием:")
result12 = s12.groupby(s12.index).sum()
print(result12)

```

Проверка уникальности индексов: False

Исходный Series:

```

A    10
B    20
A    30
C    40
D    50
B    60
dtype: int64

```

Группировка с суммированием:

```

A    40
B    80
C    40
D    50
dtype: int64

```

Рисунок 30. Задание №12

```

s13 = pd.Series([100, 200, 300], index=['2024-03-10', '2024-03-11', '
print("Исходный тип индекса:", type(s13.index))
s13.index = pd.to_datetime(s13.index)
print("После преобразования:", type(s13.index))
print(s13)

```

```

Исходный тип индекса: <class 'pandas.Index'>
После преобразования: <class 'pandas.DatetimeIndex'>
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64

```

Рисунок 31. Задание №13

```
|: data = {
    'Дата': ['2024-03-01', '2024-03-02', '2024-03-03', '2024-03-04',
    'Цена': [100, 110, 105, 120, 115]
}
df = pd.DataFrame(data)
df.to_csv('data.csv', index=False, encoding='utf-8')
print("Файл data.csv создан")

df_read = pd.read_csv('data.csv')

s14 = pd.Series(df_read['Цена'].values, index=df_read['Дата'])
print("\nПолученный Series:")
print(s14)
```

Файл data.csv создан

Полученный Series:

Дата

2024-03-01 100

2024-03-02 110

2024-03-03 105

2024-03-04 120

2024-03-05 115

dtype: int64

Рисунок 32. Задание №14

```

dates = pd.date_range(start='2024-03-01', periods=30, freq='D')
s15 = pd.Series(np.random.randint(50, 151, 30), index=dates)
plt.figure(figsize=(12, 6))
plt.plot(s15.index, s15.values, marker='o', linestyle='-',
         linewidth=2, markersize=4)
plt.title('Значения за март 2024 года', fontsize=14)
plt.xlabel('Дата', fontsize=12)
plt.ylabel('Значение', fontsize=12)
plt.grid(True)
plt.show()

```



Рисунок 33. Задание №15

3.Выполнили индивидуальное задание согласно варианту:

Создайте Series, где индексами будут даты с 1 по 15 мая 2024 года, а значениями – случайные температуры от -10 до +25 градусов. Преобразуйте индекс в DatetimeIndex, найдите разницу температур между днями (diff()), выделите дни с изменением больше 5 градусов и постройте график с отображением таких скачков.

```

: dates = pd.date_range(start='2024-05-01', periods=15, freq='D')
np.random.seed(42)
temperatures = np.random.randint(-10, 26, 15)
s = pd.Series(temperatures, index=dates)
print(s)
s.index = pd.to_datetime(s.index)
print("Тип индекса:", type(s.index))
print()
temp_diff = s.diff()
print("Разница температур между днями:")
print(temp_diff)
print()
anomaly_days = temp_diff[abs(temp_diff) > 5]
print("Дни с изменением температуры > 5 градусов:")
print(anomaly_days)
print()
plt.figure(figsize=(12, 6))
plt.plot(s.index, s.values, marker='o', linestyle='-', linewidth=2,
         markersize=6, color='blue', label='Температура')
for date in anomaly_days.index:
    plt.scatter(date, s[date], color='red', s=100, zorder=5)
    prev_date = date - pd.Timedelta(days=1)
    if prev_date in s.index:
        plt.annotate('', xy=(date, s[date]),
                     xytext=(prev_date, s[prev_date]),
                     arrowprops=dict(arrowstyle='->', color='red',
                                     lw=2))
plt.title('Температура с 1 по 15 мая 2024 года\n(красные точки - аномалии)')
plt.xlabel('Дата')
plt.ylabel('Температура')
plt.grid(True)
plt.legend()
plt.show()

```

Рисунок 34. Индивидуальное задание №1

```
2024-05-01    18
2024-05-02     4
2024-05-03    -3
2024-05-04    10
2024-05-05     8
2024-05-06    12
2024-05-07     0
2024-05-08     0
2024-05-09    13
2024-05-10    25
2024-05-11    13
2024-05-12    -8
2024-05-13    11
2024-05-14    -9
2024-05-15    13
Freq: D, dtype: int32
Тип индекса: <class 'pandas.DatetimeIndex'>

Разница температур между днями:
2024-05-01    NaN
2024-05-02   -14.0
2024-05-03    -7.0
2024-05-04    13.0
2024-05-05    -2.0
2024-05-06     4.0
2024-05-07   -12.0
2024-05-08     0.0
2024-05-09    13.0
2024-05-10    12.0
2024-05-11   -12.0
2024-05-12   -21.0
2024-05-13    19.0
2024-05-14   -20.0
2024-05-15    22.0
Freq: D, dtype: float64

Дни с изменением температуры > 5 градусов:
2024-05-02   -14.0
2024-05-03    -7.0
2024-05-04    13.0
2024-05-07   -12.0
2024-05-09    13.0
2024-05-10    12.0
2024-05-11   -12.0
2024-05-12   -21.0
2024-05-13    19.0
2024-05-14   -20.0
2024-05-15    22.0
dtype: float64
```

Рисунок 35. Индивидуальное задание №1

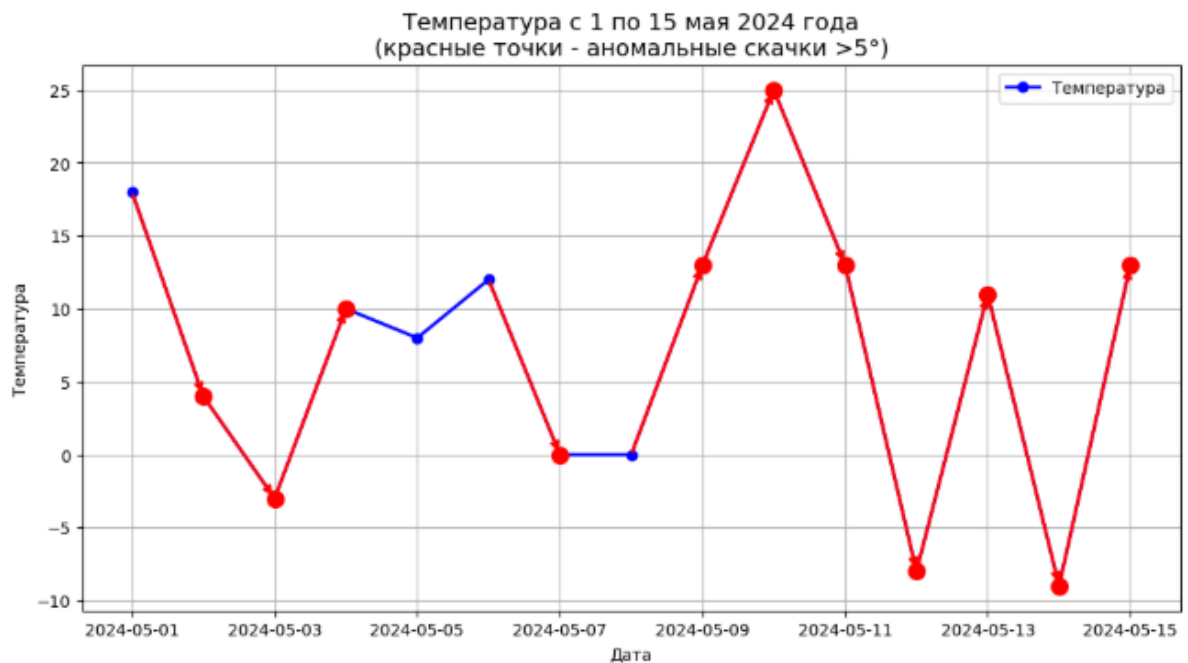


Рисунок 36. Индивидуальное задание №1

Контрольные вопросы:

1. Что такое `pandas.Series` и чем она отличается от списка в Python? `Series` - это одномерный массив данных с метками, которые называются индексами. Отличие от списка в том, что у каждого элемента `Series` есть своя метка (индекс), а в списке элементы доступны только по числовой позиции.
2. Какие типы данных можно использовать для создания `Series`? Можно использовать числа целые и дробные, строки, логические значения `True` и `False`, даты и время, а также массивы `NumPy`.
3. Как задать индексы при создании `Series`? При создании `Series` нужно передать параметр `index` со списком меток. Например: `pd.Series([10, 20, 30], index=['a', 'b', 'c'])`.
4. Каким образом можно обратиться к элементу `Series` по его индексу? Нужно написать имя `Series` и в квадратных скобках указать метку индекса. Например: `s['a']`.

5. В чём разница между `.iloc[]` и `.loc[]` при индексации Series? `.loc[]` работает с метками индексов, а `.iloc[]` работает с числовыми позициями. `.loc['a']` вернёт элемент с меткой 'a', а `.iloc[0]` вернёт первый элемент независимо от его метки.

6. Как использовать логическую индексацию в Series? Нужно внутри квадратных скобок написать условие. Например: `s[s > 10]` вернёт только те элементы, которые больше 10.

7. Какие методы можно использовать для просмотра первых и последних элементов Series? Метод `.head()` показывает первые несколько элементов, а `.tail()` показывает последние несколько элементов. По умолчанию они показывают 5 элементов, но можно указать любое число.

8. Как проверить тип данных элементов Series? Нужно использовать свойство `.dtype`. Например: `s.dtype` покажет тип данных всех элементов Series.

9. Каким способом можно изменить тип данных Series? Нужно использовать метод `.astype()`. Например: `s.astype('float')` превратит целые числа в дробные.

10. Как проверить наличие пропущенных значений в Series? Можно использовать метод `.isna()` или `.isnull()`. Они возвращают True там, где есть пропуски. А метод `.hasnans` показывает, есть ли хотя бы один пропуск.

11. Какие методы используются для заполнения пропущенных значений в Series? Основной метод - `.fillna()`. Он заменяет пропуски на указанное значение. Например, `s.fillna(0)` заменит все пропуски на ноль.

12. Чем отличается метод `.fillna()` от `.dropna()`? `.fillna()` заменяет пропущенные значения на какое-то другое значение, сохраняя все

элементы. `.dropna()` полностью удаляет строки с пропущенными значениями.

13. Какие математические операции можно выполнять с Series? Можно складывать, вычитать, умножать, делить Series друг на друга или на обычные числа. Также доступны возведение в степень, вычисление квадратного корня и другие операции.

14. В чём преимущество векторизованных операций по сравнению с циклами Python? Векторизованные операции работают сразу со всеми элементами одновременно, что делает код короче и значительно быстрее, особенно на больших данных. Циклы обрабатывают элементы по одному и работают медленнее.

15. Как применять пользовательскую функцию к каждому элементу Series? Нужно использовать метод `.apply()`. Например, `s.apply(lambda x: x * 2)` умножит каждый элемент на 2.

16. Какие агрегирующие функции доступны в Series? Доступны функции: `.sum()` для суммы, `.mean()` для среднего, `.min()` для минимума, `.max()` для максимума, `.count()` для количества элементов, `.std()` для стандартного отклонения.

17. Как узнать минимальное, максимальное, среднее и стандартное отклонение Series? Используйте методы: `s.min()`, `s.max()`, `s.mean()`, `s.std()`.

18. Как сортировать Series по значениям и по индексам? `.sort_values()` сортирует по значениям, а `.sort_index()` сортирует по индексам.

19. Как проверить, являются ли индексы Series уникальными? Нужно использовать свойство `.index.is_unique`. Оно вернёт True, если все индексы разные, и False, если есть повторяющиеся.

20. Как сбросить индексы Series и сделать их числовыми? Нужно использовать метод `.reset_index()`. Он превращает текущие индексы в обычный столбец и создаёт новые числовые индексы от 0.

21. Как можно задать новый индекс в Series? Нужно присвоить новый список меток свойству `.index`. Например: `s.index = ['x', 'y', 'z']`.

22. Как работать с временными рядами в Series? Временные ряды создаются с помощью `pd.date_range()` для создания дат. Затем эти даты передаются как индекс Series. После этого можно выбирать данные по датам, использовать срезы по датам и применять специальные методы для работы с временными данными.

23. Как преобразовать строковые даты в формат DatetimeIndex? Нужно использовать `pd.to_datetime()`. Например: `s.index = pd.to_datetime(s.index)`. Это превращает строки с датами в специальный тип данных для работы с датами.

24. Каким образом можно выбрать данные за определённый временной диапазон? Можно использовать срез по датам: `s['2024-03-01':'2024-03-10']` выберет все данные с 1 по 10 марта.

25. Как загрузить данные из CSV-файла в Series? Сначала читаем весь файл через `pd.read_csv()` в DataFrame, затем выбираем нужный столбец. Например: `df = pd.read_csv('file.csv'); s = df['название_столбца']`.

26. Как установить один из столбцов CSV-файла в качестве индекса Series? При чтении файла можно использовать параметр `index_col`: `df = pd.read_csv('file.csv', index_col='название_столбца')`. Затем извлечь нужный столбец в Series.

27. Для чего используется метод `.rolling().mean()` в Series? Этот метод вычисляет скользящее среднее. Он берёт окно из нескольких элементов и

считает среднее по этому окну, затем окно сдвигается. Это помогает сгладить колебания данных и увидеть общий тренд.

28. Как работает метод `.pct_change()`? Какие задачи он решает? `.pct_change()` вычисляет процентное изменение между текущим и предыдущим элементом. Он показывает, на сколько процентов выросло или упало значение по сравнению с прошлым периодом. Это полезно для расчёта доходности или темпов роста.

29. В каких ситуациях полезно использовать `.rolling()` и `.pct_change()`? `.rolling()` полезен для сглаживания шумов на графиках и поиска трендов в данных. `.pct_change()` полезен для расчёта доходности акций, темпов роста продаж, инфляции и любых других данных, где важно относительное изменение.

30. Почему NaN могут появляться в Series, и как с ними работать? NaN появляются, когда при сложении двух Series индексы не совпадают, при чтении данных с пустыми ячейками в файле, или при делении на ноль. Работать с ними можно так: `.isna()` для поиска пропусков, `.fillna()` для замены пропусков на нужное значение, `.dropna()` для удаления строк с пропусками.

Вывод: в ходе выполнения заданий были изучены основные возможности `pandas.Series`. Освоено создание Series с пользовательскими индексами, доступ к элементам через `loc` и `iloc`, а также логическая фильтрация данных. Изучены методы `head` и `tail` для просмотра данных, проверка и изменение типов через `dtype` и `astype`. Рассмотрена работа с пропущенными значениями: их обнаружение через `isna`, заполнение через `fillna` и удаление через `dropna`. Освоены арифметические операции, применение функций через `apply` и основные статистические методы `sum`, `mean`, `min`, `max`, `std`. Изучена работа с временными рядами: создание

индексов из дат, выборка по диапазону и преобразование строк в `DatetimeIndex`. Также была рассмотрена загрузка данных из CSV и построение графиков с помощью `matplotlib`. В индивидуальном задании закреплены навыки работы с `diff` для поиска аномалий и их визуализации. В результате получены практические навыки анализа одномерных данных с помощью `pandas.Series`.